

Guion para el video de entrega

Duración máxima: 15 minutos. Graba con OBS Studio o QuickTime (Mac: Cmd+Shift+5 → "Grabar toda la pantalla" o una ventana).

Este guion está escrito como **texto para leer en voz alta** mientras grabas, con vocabulario técnico. Las líneas en cursiva indican qué mostrar en pantalla en ese momento.

1. Introducción (30 s)

(Muestra el proyecto abierto en el editor)

"Buenas, en este video voy a presentar el proyecto eshop-services, una arquitectura de microservicios desarrollada en .NET 9 siguiendo el patrón CQRS, es decir, separación entre comandos y consultas. Voy a explicar la arquitectura del código, mostrar el catálogo de productos funcionando en producción, y el proceso paso a paso de cómo publiqué la base de datos, la API y el frontend en la nube."

2. Arquitectura y código (2-3 min)

(Abre el editor y muestra los archivos mientras hablas)

"El proyecto tiene dos microservicios. Catalog.API expone un API REST para la gestión de productos: búsqueda por nombre, filtrado por categoría y rango de precio, inserción, actualización, eliminación y consulta paginada. Basket.API gestiona el carrito de compras."

(Abre `GetProductsEndPoint.cs` y `GetProductsQueryHandler.cs`)

"Cada microservicio expone sus endpoints usando Carter, una librería de enrutamiento sobre Minimal APIs de ASP.NET Core, y la lógica de negocio se despacha con MediatR, implementando el patrón mediador: cada operación es un Command o una Query con su respectivo Handler. Este endpoint de consulta de productos acepta filtros combinables por nombre, categoría y rango de precio, todos opcionales, y devuelve el resultado paginado. La persistencia se maneja con Marten, un ORM de tipo document database sobre PostgreSQL: los objetos .NET se serializan directamente a columnas JSONB, sin necesidad de mapeo relacional explícito ni migraciones manuales."

(Abre `CacheBasketRepository.cs` y `Program.cs` de Basket.API)

"Basket.API implementa el patrón decorador para el manejo de caché: la clase `BasketRepository` accede directamente a PostgreSQL, y `CacheBasketRepository` la envuelve agregando una capa de caché distribuida con Redis, a través de la interfaz estándar `IDistributedCache` de .NET. Esto se registra en el contenedor de inyección de dependencias con el método `Decorate`, de la librería Scrutor, sin modificar el contrato de la interfaz `IBasketRepository`. Con esto, el servicio es completamente compatible con Redis como caché distribuida: las lecturas del carrito primero consultan Redis, y solo si no hay dato en caché se consulta la base relacional, reduciendo la carga sobre PostgreSQL."

(Abre `App.vue`)

"El frontend está construido con Vue 3, usando la Composition API y `<script setup>`, y Vite como herramienta de build. Consume la API mediante `fetch`, con manejo reactivo de estado para el catálogo, los

filtros y la paginación."

3. Demo del catálogo en producción (3 min)

(Abre en el navegador la URL real de Netlify, por ejemplo <https://alexisbbya.netlify.app>)

"Ahora una demostración funcional completa, directamente contra el entorno de producción: el sitio publicado en Netlify, que consume la API publicada en Render, que a su vez persiste en la base de datos publicada en Neon."

(Inserción)

"Inserción de un producto nuevo: nombre, descripción, categorías, imagen y precio."

(Búsqueda por nombre)

"Búsqueda por nombre."

(Filtro combinado)

"Filtro combinado por categoría y rango de precio, aplicados simultáneamente."

(Actualización)

"Actualización de un producto existente."

(Eliminación)

"Eliminación por nombre."

(Paginación)

"Y la consulta paginada: el listado completo se pagina desde el backend con los parámetros `pageIndex` y `pageSize`."

4. Publicar la base de datos: Neon (2-3 min)

(Navegador: neon.tech)

"Para publicar la base de datos en un dominio propio en la nube, usé Neon, un servicio de PostgreSQL serverless. El proceso fue el siguiente."

(Muestra el dashboard / proyecto)

"Primero, creé una cuenta en Neon e inicié un nuevo proyecto, indicando el nombre y la región más cercana."

(Muestra la pantalla de "Connect")

"Desde el botón 'Connect' del proyecto se obtiene el connection string de conexión a PostgreSQL, con el host, usuario, contraseña y nombre de base de datos —oculto la contraseña por seguridad—. Ese host,

terminado en `neon.tech`, es justamente el dominio en la nube de la base de datos."

(Opcional: muestra el SQL Editor)

"Neon también incluye un editor SQL directo desde el navegador, útil para crear bases de datos adicionales o ejecutar consultas sin necesidad de un cliente externo."

"Ese connection string no se coloca directamente en el código fuente, sino como variable de entorno en el servicio donde se aloja la API, que es el siguiente paso."

5. Publicar la API: Render (3 min)

(Navegador: GitHub, mostrando el repositorio)

"El código está versionado en un repositorio de GitHub. Para publicar la API en la nube usé Render, conectado directamente a ese repositorio."

(Navegador: render.com, creación del Web Service)

"Los pasos fueron: crear una cuenta en Render y autorizar el acceso al repositorio; luego, crear un nuevo 'Web Service', seleccionando el repositorio del proyecto."

(Muestra la configuración: runtime Docker, Dockerfile path)

"En la configuración indiqué que el runtime es Docker, y la ruta del Dockerfile correspondiente a Catalog.API dentro del repositorio."

(Muestra las variables de entorno)

"Luego configuré las variables de entorno: el connection string de Neon para la base de datos, el origen permitido para CORS —que es el dominio de Netlify—, y el entorno de ejecución en modo Production."

(Muestra el log de deploy terminando en éxito y la URL pública)

"Al desplegar, Render construye la imagen Docker automáticamente y publica el servicio en una URL propia sobre HTTPS. Cada vez que se hace un nuevo `git push` al repositorio, Render reconstruye y despliega automáticamente, sin pasos manuales adicionales."

(Terminal o navegador: curl https://tu-api.onrender.com/products)

"Y aquí confirmo que la API responde correctamente desde su URL pública."

6. Publicar el frontend: Netlify (2-3 min)

(Navegador: netlify.com)

"Para el frontend usé Netlify. El proceso: crear una cuenta, conectarla a GitHub, e importar el mismo repositorio del proyecto."

(Muestra la configuración de build)

"En la configuración de build indiqué el directorio `frontend` como base, el comando `npm run build`, que ejecuta Vite, y el directorio de salida `dist` como directorio de publicación."

(Muestra la variable de entorno VITE_API_URL)

"La URL de la API publicada en Render se inyecta en tiempo de compilación mediante la variable de entorno `VITE_API_URL`, para que el frontend sepa a qué servidor hacer las peticiones."

(Muestra el deploy terminado y la URL pública)

"Al desplegar, Netlify genera una URL pública propia, y también aquí cada nuevo `git push` dispara un despliegue automático."

7. Cierre (30 s)

"En resumen: la base de datos está publicada en Neon con su propio dominio, la API de catálogo está publicada en Render y expone búsqueda, filtros combinados, operaciones CRUD completas y paginación, el frontend está publicado en Netlify consumiendo esa API en producción, y el microservicio de carrito, Basket.API, implementa una capa de caché completamente compatible con Redis mediante el patrón decorador. Gracias."

Checklist antes de grabar

- ☐ `git push` hecho y repo visible en GitHub.
- ☐ Servicio en Render desplegado y respondiendo (probar `/products` antes de grabar).
- ☐ Sitio en Netlify desplegado y probado end-to-end (crear/buscar/filtrar/actualizar/eliminar/paginar).
- ☐ `Cors__AllowedOrigins` en Render actualizado con la URL final de Netlify.
- ☐ Contraseñas/connection strings tapadas si el video se va a compartir públicamente.
- ☐ Practica el guion en voz alta una vez, cronometrado, para que quede bajo 15 min y no suene leído palabra por palabra.